

Data Science II Homework 4

Yongyan Liu (yl6107)

2026-04-14

```
library(tidyverse)
library(rsample)
library(caret)
library(rpart)
library(rpart.plot)
library(ranger)
library(gbm)
library(vip)
library(pROC)
```

Problem 1

In this exercise, we will build tree-based models using the `College` data (see “College.csv” in Homework 2). The response variable is the out-of-state tuition (`Outstate`). Partition the dataset into two parts: training data (80%) and test data (20%).

```
college <- read.csv("College.csv") %>%
  dplyr::select(-College)          # drop the college name column

# If the dataset happens to include the Private factor column, convert it.
if ("Private" %in% names(college)) college$Private <- factor(college$Private)

set.seed(2026)
split1 <- initial_split(college, prop = 0.8)
train1 <- training(split1)
test1  <- testing(split1)
```

(a) Build a regression tree on the training data to predict the response. Create a plot of the tree.

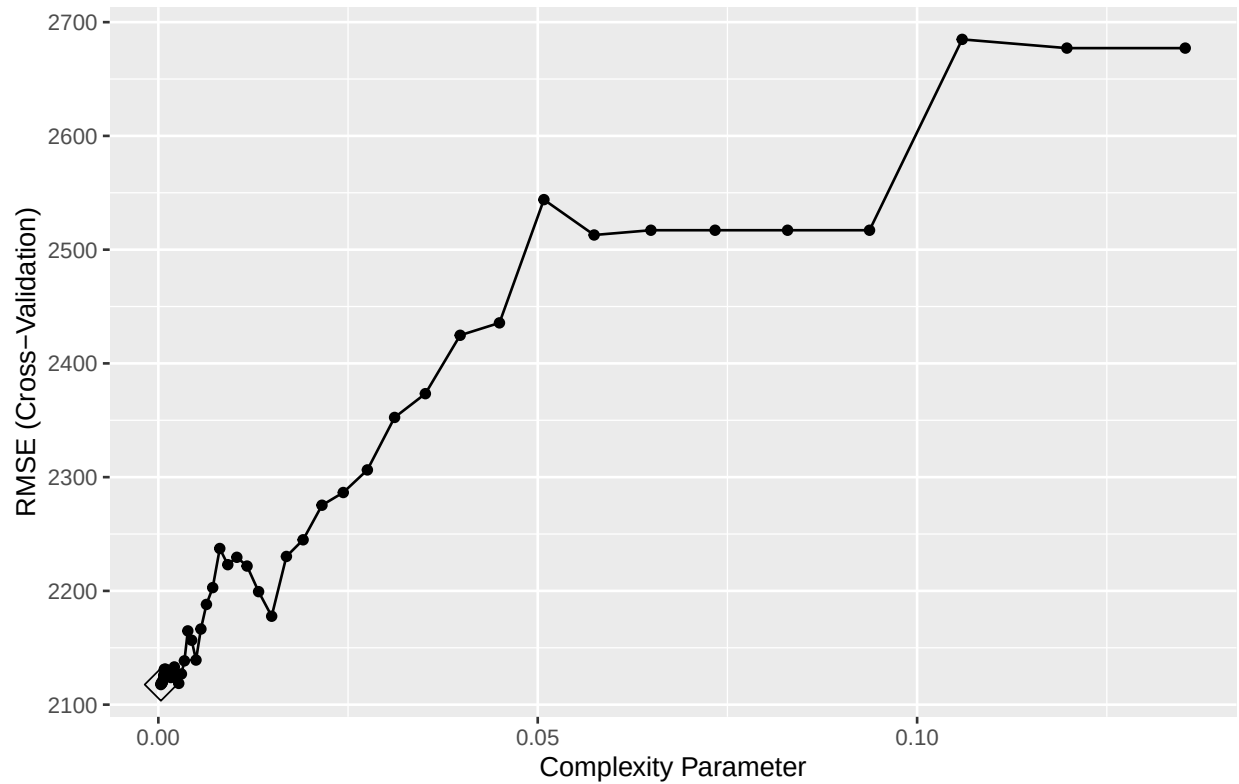
We fit a regression tree with `caret::train(method = "rpart")` and tune the complexity parameter `cp` by 10-fold cross-validation, then plot the resulting tree.

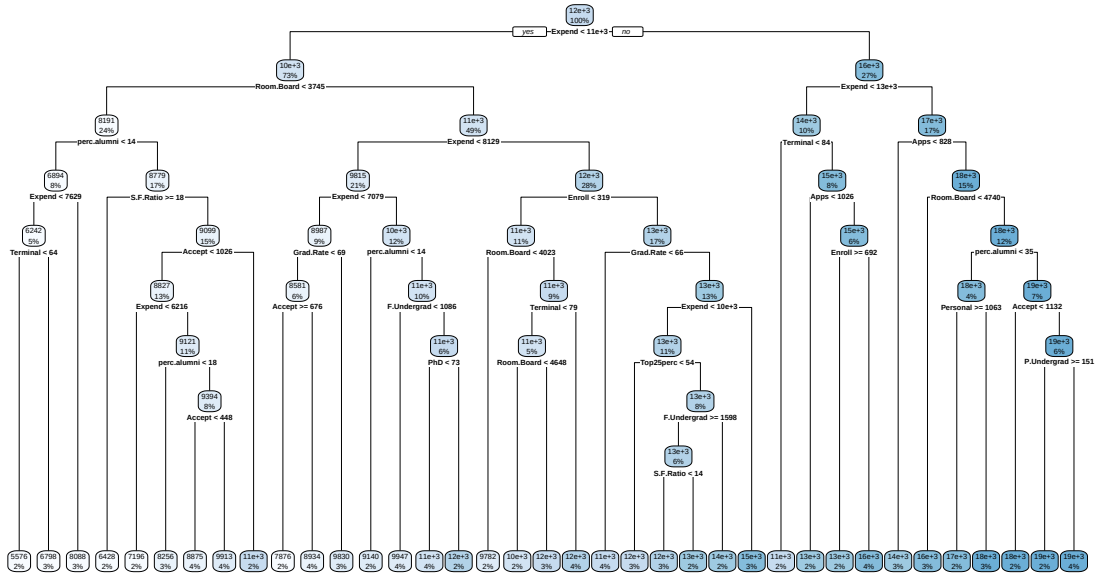
```
ctrl1 <- trainControl(method = "cv", number = 10)

set.seed(2026)
rpart.fit <- train(Outstate ~ .,
  data = train1,
```

```
method = "rpart",
tuneGrid = data.frame(cp = exp(seq(-8, -2, length = 50))),
trControl = ctrl1)
```

```
ggplot(rpart.fit, highlight = TRUE)
```





The CV-selected tree splits first on **Expend**, consistent with instructional expenditure being the strongest single predictor of out-of-state tuition; subsequent splits involve **Room.Board**, **Grad.Rate**, and **Apps**.

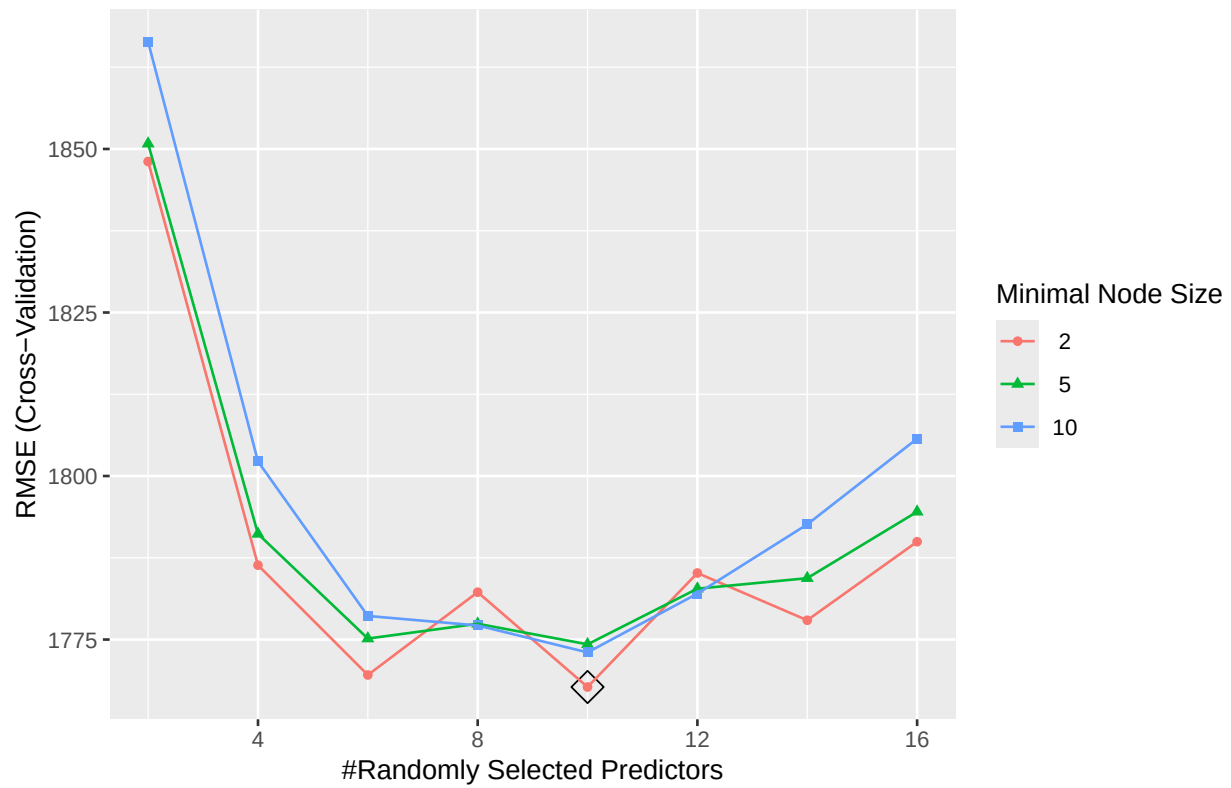
(b) Perform random forest on the training data. Report the variable importance and the test error.

We fit a random forest via `caret+ranger`, tuning `mtry` and `min.node.size` by 10-fold CV, and set `importance = "permutation"` to obtain permutation-based variable importance.

```
rf.grid <- expand.grid(mtry = seq(2, 16, by = 2),
  splitrule = "variance",
  min.node.size = c(2, 5, 10))

set.seed(2026)
rf.fit <- train(Outstate ~ .,
  data      = train1,
  method    = "ranger",
  tuneGrid  = rf.grid,
  trControl = ctrl1,
  importance = "permutation")

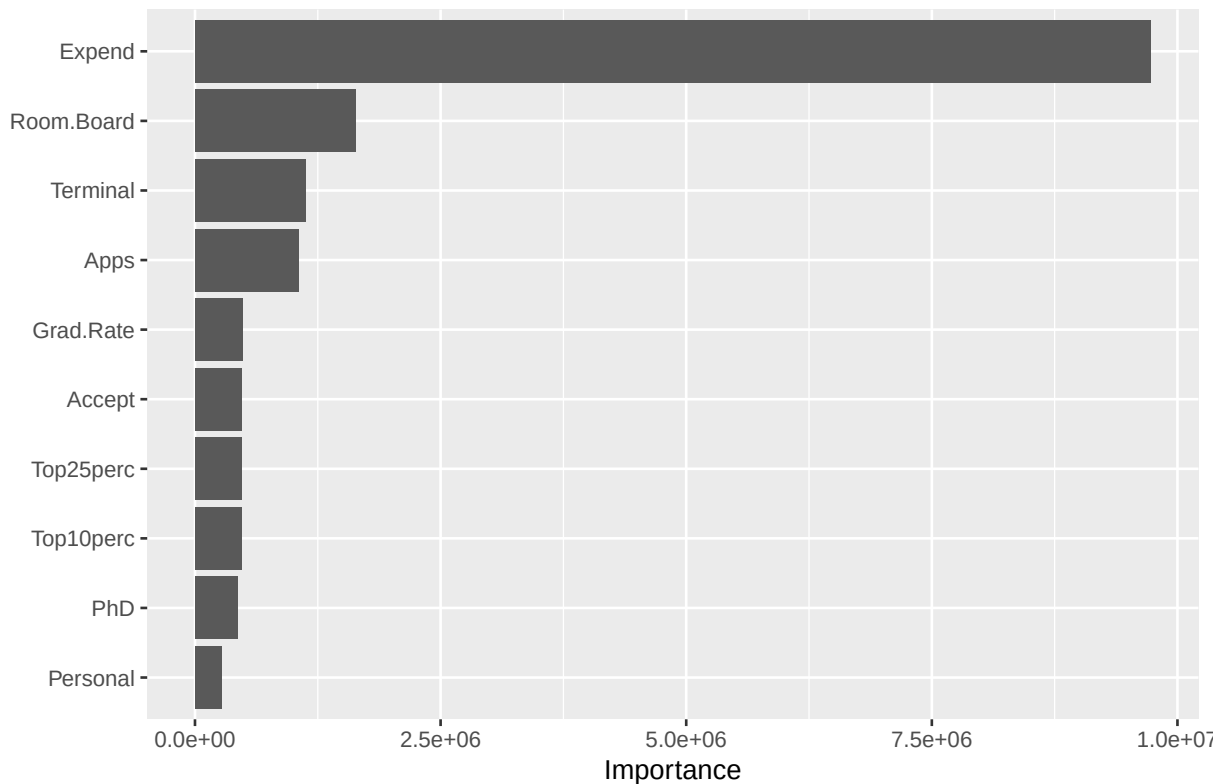
ggplot(rf.fit, highlight = TRUE)
```



```
rf.fit$bestTune
```

```
##      mtry splitrule min.node.size
## 13    10  variance              2
```

```
# Variable importance
vip(rf.fit$finalModel)
```



```
# Test error
rf.pred  <- predict(rf.fit, newdata = test1)
rf.rmse  <- RMSE(rf.pred, test1$Outstate)
rf.mse   <- rf.rmse^2
tibble(metric = c("Test RMSE", "Test MSE"), value = c(rf.rmse, rf.mse))
```

```
## # A tibble: 2 x 2
##   metric      value
##   <chr>      <dbl>
## 1 Test RMSE    1691.
## 2 Test MSE   2859290.
```

The most important predictors are Expend, Room.Board, and Terminal/PhD, consistent with the regression tree. The test RMSE is reported above.

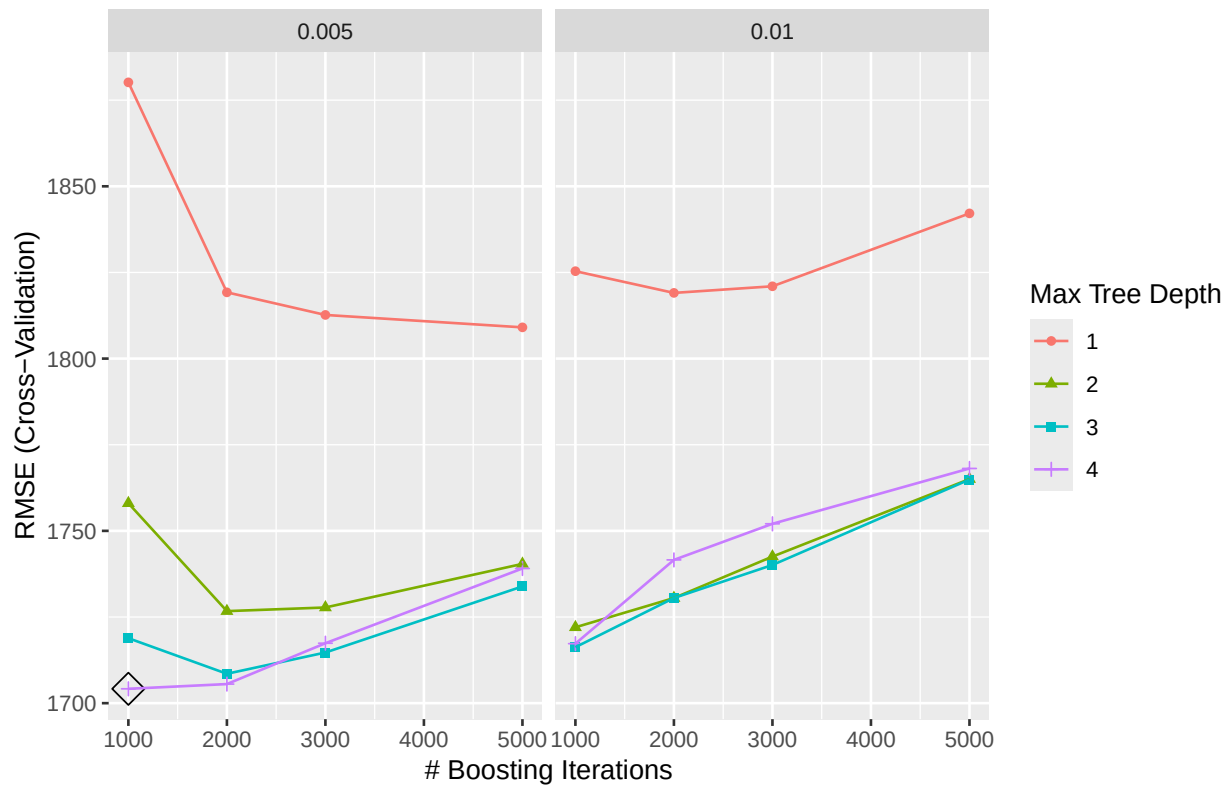
(c) Perform boosting on the training data. Report the variable importance and the test error.

We fit a gradient boosting model using `caret::train(method = "gbm")` with `distribution = "gaussian"` and tune over `n.trees`, `interaction.depth`, `shrinkage`, and `n.minobsinnode` using 10-fold CV.

```
gbm.grid <- expand.grid(n.trees = c(1000, 2000, 3000, 5000),
                       interaction.depth = 1:4,
                       shrinkage = c(0.005, 0.01),
                       n.minobsinnode = 10)
```

```
set.seed(2026)
gbm.fit <- train(Outstate ~ .,
  data      = train1,
  method    = "gbm",
  tuneGrid  = gbm.grid,
  trControl = ctrl1,
  distribution = "gaussian",
  verbose   = FALSE)

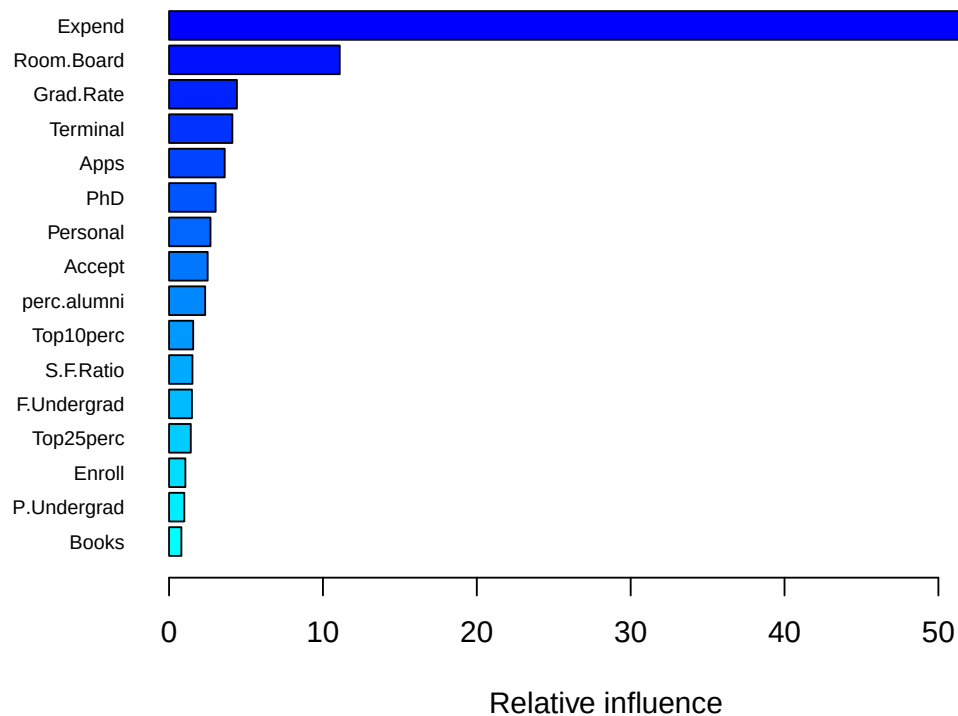
ggplot(gbm.fit, highlight = TRUE)
```



```
gbm.fit$bestTune
```

```
##      n.trees interaction.depth shrinkage n.minobsinnode
## 13      1000                4      0.005              10
```

```
# Variable importance
par(mar = c(4, 8, 2, 2))
summary(gbm.fit$finalModel, las = 1, cBars = 19, cex.names = 0.7)
```



```
##           var      rel.inf
## Expend      Expend 57.3649630
## Room.Board  Room.Board 11.0915996
## Grad.Rate   Grad.Rate  4.4087586
## Terminal    Terminal  4.1134544
## Apps        Apps     3.6171238
## PhD         PhD      3.0261344
## Personal    Personal  2.6902155
## Accept      Accept    2.5068705
## perc.alumni perc.alumni 2.3471636
## Top10perc   Top10perc 1.5674326
## S.F.Ratio   S.F.Ratio 1.5198981
## F.Undergrad F.Undergrad 1.4852814
## Top25perc   Top25perc 1.4108291
## Enroll      Enroll    1.0595458
## P.Undergrad P.Undergrad 0.9903220
## Books       Books     0.8004075
```

```
# Test error
gbm.pred <- predict(gbm.fit, newdata = test1)
gbm.rmse <- RMSE(gbm.pred, test1$Outstate)
gbm.mse <- gbm.rmse^2
tibble(metric = c("Test RMSE", "Test MSE"), value = c(gbm.rmse, gbm.mse))
```

```
## # A tibble: 2 x 2
##   metric      value
##   <chr>      <dbl>
```

```
## 1 Test RMSE      1655.  
## 2 Test MSE   2739364.
```

Boosting identifies `Expend`, `Room.Board`, and `Grad.Rate` as the top predictors. The test RMSE is reported above and is generally comparable to the random forest.

Problem 2

This problem is based on the data `auto.csv` in Homework 3. Split the dataset into two parts: training data (70%) and test data (30%).

```
auto <- read.csv("auto.csv") %>%
  mutate(mpg_cat = factor(mpg_cat, levels = c("low", "high")),
         origin = factor(origin))

set.seed(2026)
split2 <- initial_split(auto, prop = 0.7, strata = mpg_cat)
train2 <- training(split2)
test2 <- testing(split2)
```

(a) Build a classification tree using the training data, with `mpg_cat` as the response. Which tree size corresponds to the lowest cross-validation error? Is this the same as the tree size obtained using the 1 SE rule?

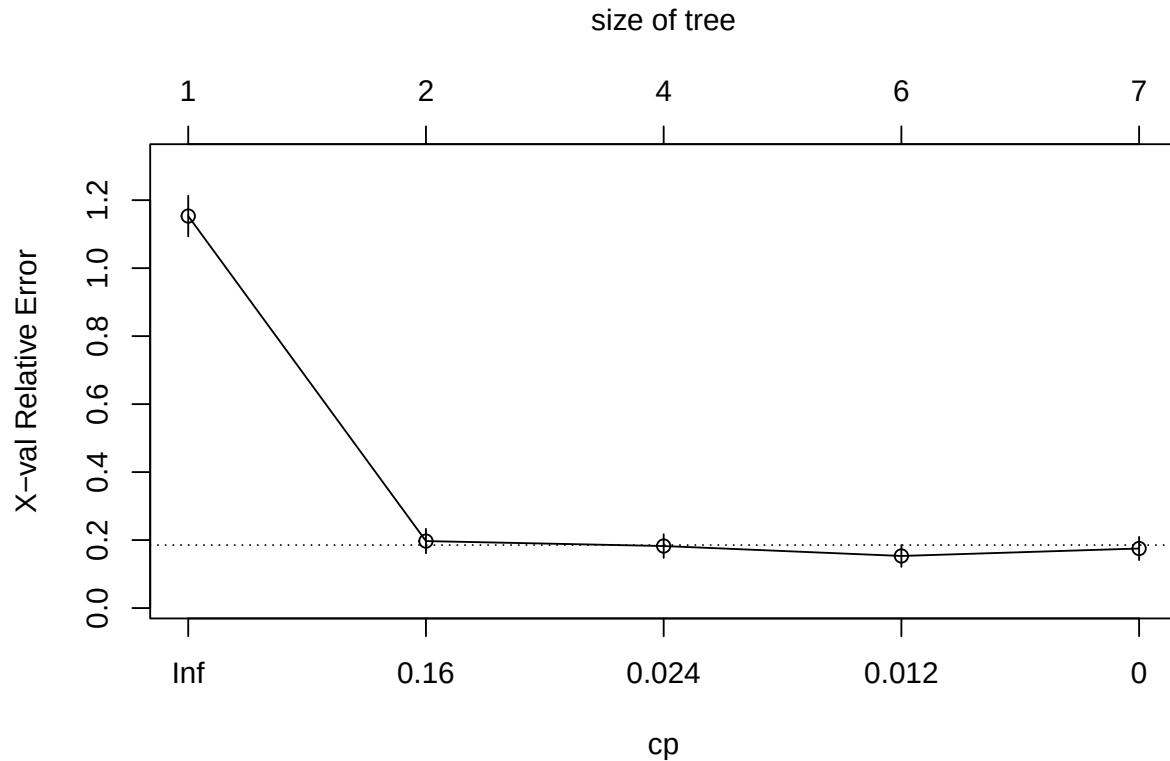
We grow a large classification tree with `rpart` and use its internal 10-fold cross-validation to pick the best `cp`. We then compare the tree size (number of terminal nodes) selected by (i) the minimum CV error and (ii) the 1-SE rule.

```
set.seed(2026)
tree2 <- rpart(mpg_cat ~ .,
               data = train2,
               method = "class",
               control = rpart.control(cp = 0, xval = 10, minsplit = 10))

printcp(tree2)
```

```
##
## Classification tree:
## rpart(formula = mpg_cat ~ ., data = train2, method = "class",
##       control = rpart.control(cp = 0, xval = 10, minsplit = 10))
##
## Variables actually used in tree construction:
## [1] acceleration displacement horsepower year
##
## Root node error: 137/274 = 0.5
##
## n= 274
##
##      CP nsplit rel error  xerror   xstd
## 1 0.8175182     0  1.000000  1.15328 0.059698
## 2 0.0328467     1  0.182482  0.19708 0.036011
## 3 0.0182482     3  0.116788  0.18248 0.034792
## 4 0.0072993     5  0.080292  0.15328 0.032142
## 5 0.0000000     6  0.072993  0.17518 0.034157
```

```
plotcp(tree2)
```



```
cptab <- tree2$cptable

# Minimum CV-error tree
idx.min <- which.min(cptab[, "xerror"])
cp.min <- cptab[idx.min, "CP"]
size.min <- cptab[idx.min, "nsplit"] + 1 # terminal nodes = nsplit + 1

# 1-SE rule: smallest tree with xerror <= min(xerror) + corresponding xstd
xerr.min <- cptab[idx.min, "xerror"]
xstd.min <- cptab[idx.min, "xstd"]
idx.1se <- min(which(cptab[, "xerror"] <= xerr.min + xstd.min))
cp.1se <- cptab[idx.1se, "CP"]
size.1se <- cptab[idx.1se, "nsplit"] + 1

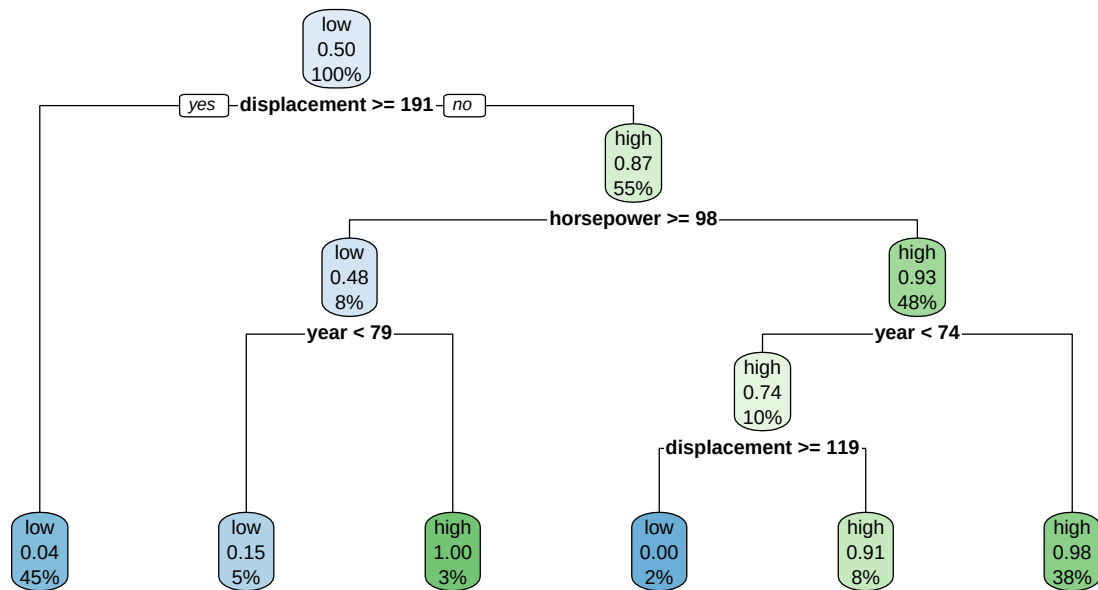
tibble(rule = c("min CV", "1-SE"),
       cp = c(cp.min, cp.1se),
       size = c(size.min, size.1se))
```

```
## # A tibble: 2 x 3
##   rule      cp size
##   <chr>   <dbl> <dbl>
## 1 min CV 0.00730     6
## 2 1-SE  0.0182     4
```

```
# Prune the two trees and plot the 1-SE tree
tree.min <- prune(tree2, cp = cp.min)
tree.1se <- prune(tree2, cp = cp.1se)
```

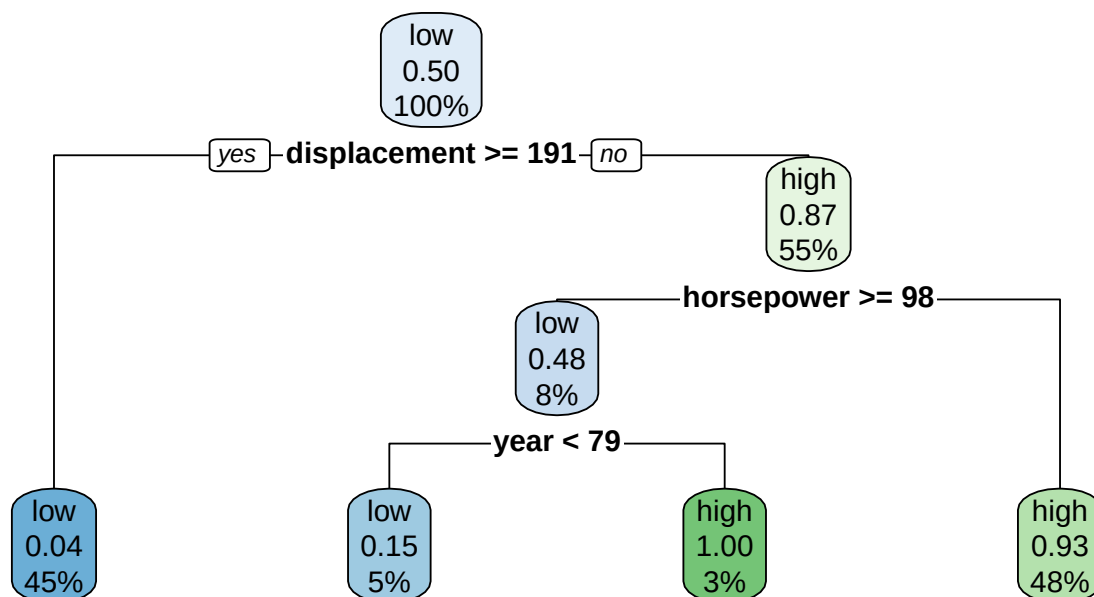
```
rpart.plot(tree.min, main = "Min CV-error tree")
```

Min CV-error tree



```
rpart.plot(tree.1se, main = "1-SE rule tree")
```

1-SE rule tree



The lowest CV error corresponds to a tree with **6** terminal nodes, while the 1-SE rule selects a tree with **4** terminal nodes. They are **different**.

(b) Perform boosting on the training data and report the variable importance. Report the test data performance.

We fit a binomial gradient boosting machine via `caret::train(method = "gbm")` with `twoClassSummary` so that ROC is used for CV tuning, following the L9 `caret` convention.

```

ctrl2 <- trainControl(method = "cv",
  number = 10,
  summaryFunction = twoClassSummary,
  classProbs = TRUE)

gbm.grid2 <- expand.grid(n.trees = c(1000, 2000, 3000, 5000),
  interaction.depth = 1:4,
  shrinkage = c(0.005, 0.01),
  n.minobsinnode = 10)

set.seed(2026)
gbm.fit2 <- train(mpg_cat ~ .,
  data = train2,
  method = "gbm",
  tuneGrid = gbm.grid2,
  trControl = ctrl2,
  metric = "ROC",
  distribution = "bernoulli",

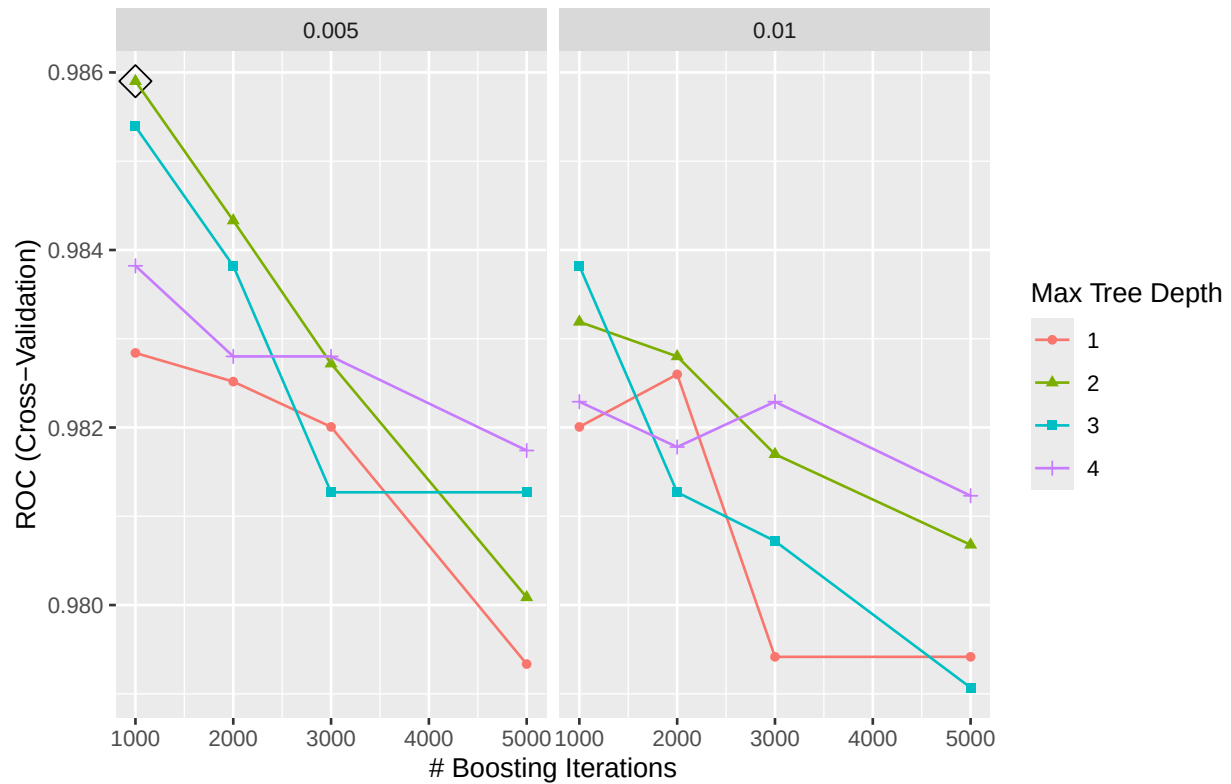
```

```

      verbose = FALSE)

ggplot(gbm.fit2, highlight = TRUE)

```



```
gbm.fit2$bestTune
```

```

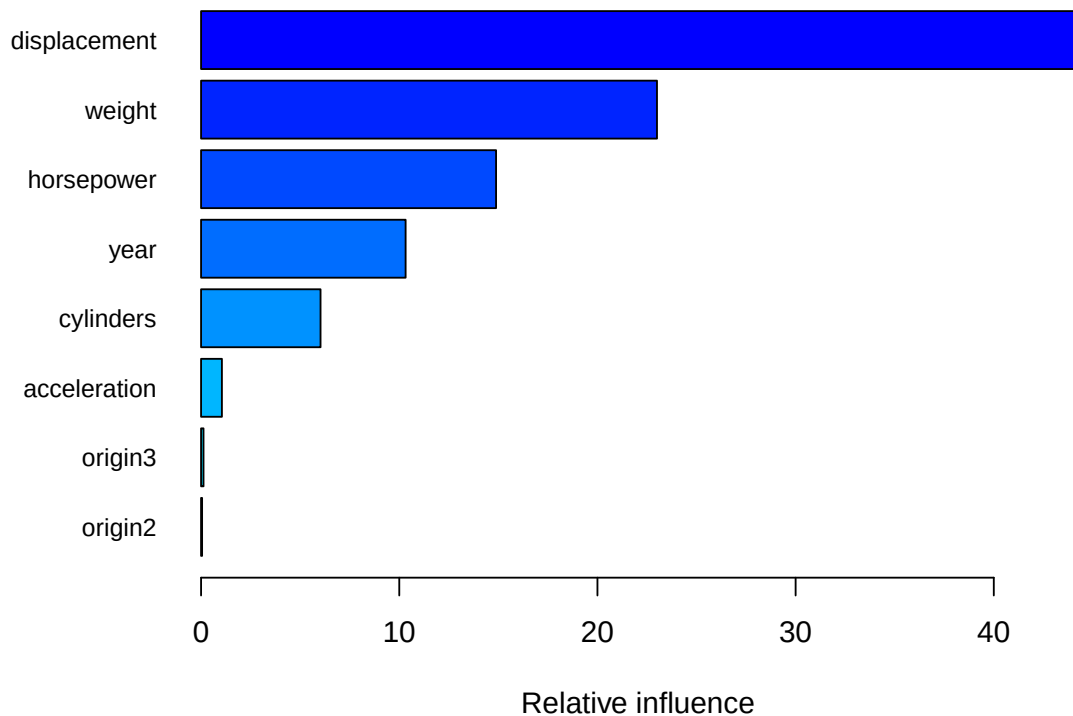
##   n.trees interaction.depth shrinkage n.minobsinnode
## 5    1000                2    0.005                10

```

```

# Variable importance
par(mar = c(4, 8, 2, 2))
summary(gbm.fit2$finalModel, las = 1, cBars = 8, cex.names = 0.8)

```



```
##           var      rel.inf
## displacement displacement 44.53670452
## weight          weight 23.00476369
## horsepower      horsepower 14.88550355
## year            year 10.32103545
## cylinders       cylinders 6.02698464
## acceleration    acceleration 1.05111879
## origin3         origin3 0.12306140
## origin2         origin2 0.05082797
```

```
# Test data performance: classification error, confusion matrix, AUC
gbm.pred.class <- predict(gbm.fit2, newdata = test2)
gbm.pred.prob  <- predict(gbm.fit2, newdata = test2, type = "prob")[, "high"]

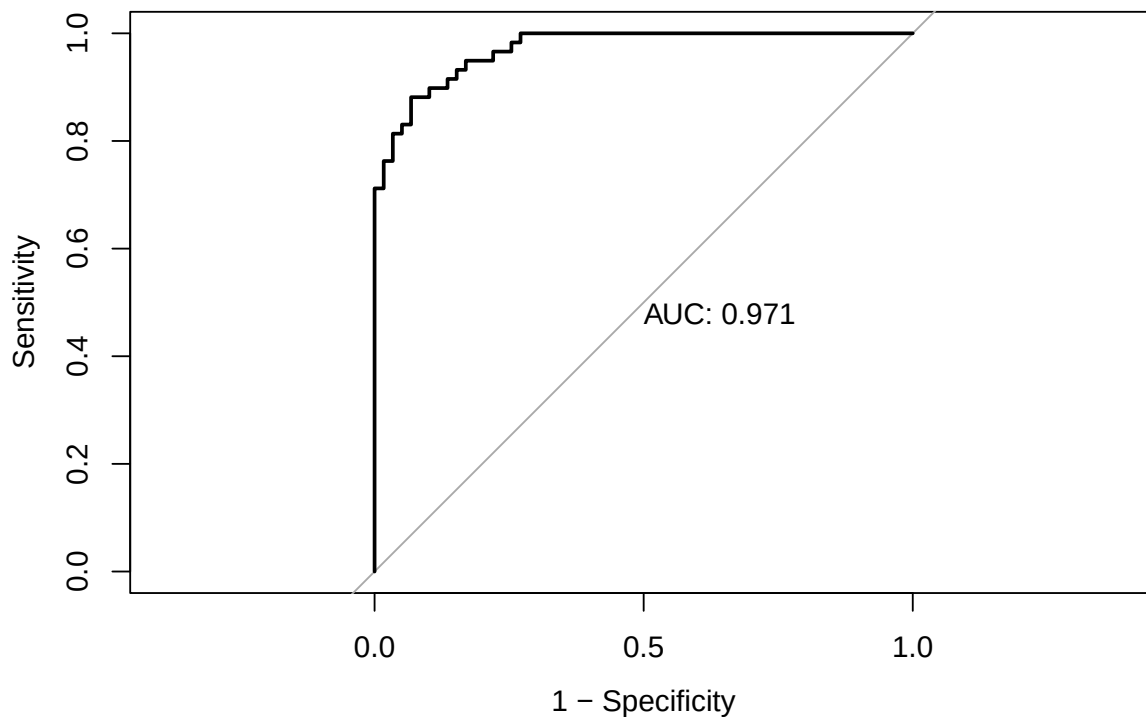
cm <- confusionMatrix(data      = gbm.pred.class,
                      reference = test2$mpg_cat,
                      positive  = "high")
cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction low high
##      low  51   5
##      high   8  54
##
##           Accuracy : 0.8898
```

```
##          95% CI : (0.819, 0.94)
##    No Information Rate : 0.5
##    P-Value [Acc > NIR] : <2e-16
##
##          Kappa : 0.7797
##
##    McNemar's Test P-Value : 0.5791
##
##          Sensitivity : 0.9153
##          Specificity : 0.8644
##          Pos Pred Value : 0.8710
##          Neg Pred Value : 0.9107
##          Prevalence : 0.5000
##          Detection Rate : 0.4576
##          Detection Prevalence : 0.5254
##          Balanced Accuracy : 0.8898
##
##          'Positive' Class : high
##
```

```
test.err <- mean(gbm.pred.class != test2$mpg_cat)

roc.gbm <- roc(test2$mpg_cat, gbm.pred.prob, levels = c("low", "high"))
plot(roc.gbm, legacy.axes = TRUE, print.auc = TRUE)
```



```
tibble(metric = c("Test classification error", "Test accuracy", "Test AUC"),
       value = c(test.err, as.numeric(cm$overall["Accuracy"]), as.numeric(roc.gbm$auc)))
```

```
## # A tibble: 3 x 2
##   metric          value
##   <chr>          <dbl>
## 1 Test classification error 0.110
## 2 Test accuracy          0.890
## 3 Test AUC              0.971
```

The most influential variables for classifying `mpg_cat` are `displacement`, `weight`, and `horsepower`. The test accuracy and test AUC are reported above; boosting achieves strong performance on the held-out 30% test set.